Oracle ORDBMS – 2

Procedurális komponensek
Öröklés · Polimorfizmus

MDB
9. gyakorlat

A

Procedurális komponensek

- Member function/procedure
- Static method
- MAP / ORDER függvény

B

Öröklés, polimorfizmus

- UNDER kulcsszó
- OVERRIDING MEMBER FUNCTION
- IS OF / TREAT operátorok

Előző óra

`CREATE TYPE ... AS OBJECT`

Alaptípus attribútumokkal

`TYPE BODY / MEMBER FUNCTION`

Tagfüggvény implementálás

`Object Table`

Sorok = objektum példányok

`VARRAY / Nested Table`

Kollekciótípusok

1. Procedurális komponensek áttekintése

Az Oracle UDT-k nem csak adatot, hanem logikát is tartalmazhatnak

MEMBER FUNCTION

Érteket ad vissza (RETURN)

```
MEMBER FUNCTION ar_kedv(
  szazalek NUMBER)
RETURN NUMBER IS
BEGIN
  RETURN SELF.ar *
    (1 - szazalek/100);
END;

-- hivas lekerdezesben:
SELECT p.ar_kedv(10)
FROM termekek p;
```

- Hívható SELECT / WHERE-ben
- read-only ((nocopy))
- Visszetérés típusa megadott

MEMBER PROCEDURE

Nem ad vissza érteket

```
MEMBER PROCEDURE
  ar_emeles(szaz NUMBER)
IS BEGIN
  SELF.ar := SELF.ar *
    (1 + szaz/100);
END;

-- hivas PL/SQL-bol:
DECLARE p termek_t;
BEGIN p.ar_emeles(15);
END;
```

- Nincs RETURN érték
- Módosíthatja az objektumot
- IN OUT NOCOPY

STATIC FUNCTION

Példány nélkül hívható

```
STATIC FUNCTION
  afa_kulcs RETURN NUMBER
IS BEGIN
  RETURN 0.27;
END;

-- hivas peldany NELKUL:
SELECT termek_t.afa_kulcs()
FROM dual;
-- nincs SELF referencia
```

- Meghívás: tipusnev.metodusnev()
- Nincs SELF (példányfüggetlen)

MAP és ORDER

Rendezéshez és egyenlőségvizsgálathoz meg kell mondani az Oracle-nek, hogy hogyan hasonlítsa össze az objektumokat

Miért kell?

Az alap Oracle objektumok NEM rendezhetőek (ORDER BY) és nem hasonlíthatóak össze (>, <). Ehhez vagy MAP, vagy ORDER függvényt kell definiálni a típusnak.
FONTOS: egyszerre csak EGY megengedett (map v. order) nem mindkettő!

MAP MEMBER FUNCTION

Skaláris értékre képezi le az objektumot rendezéshez

```
CREATE OR REPLACE TYPE termek_t AS OBJECT (
  nev VARCHAR2(100), ar NUMBER,
  MAP MEMBER FUNCTION ertek RETURN NUMBER
) NOT FINAL;
CREATE OR REPLACE TYPE BODY termek_t AS
  MAP MEMBER FUNCTION ertek
  RETURN NUMBER IS
  BEGIN RETURN SELF.ar; END;
END;
-- Ezutan mukodik:
SELECT * FROM termekek t ORDER BY VALUE(t);
```

ORDER MEMBER FUNCTION

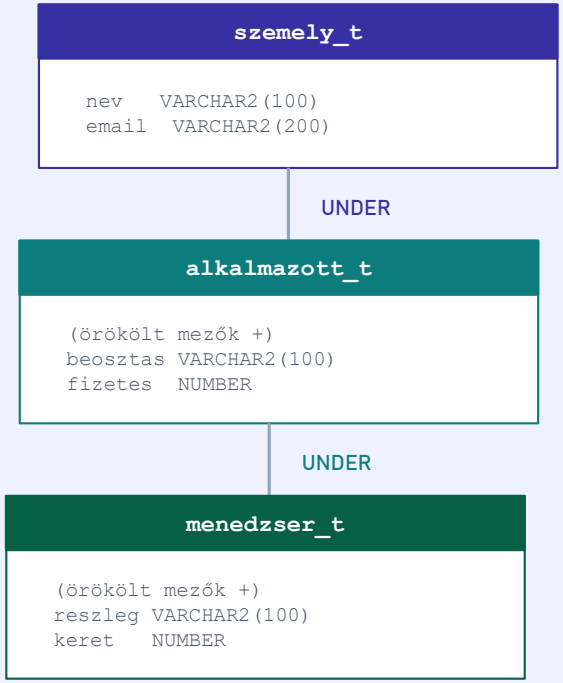
Két objektumot hasonlít össze: -1 / 0 / +1 eredmény

```
CREATE OR REPLACE TYPE termek_t AS OBJECT (
  nev VARCHAR2(100), ar NUMBER,
  ORDER MEMBER FUNCTION
    osszehasonlit(masik termek_t) RETURN INTEGER
) NOT FINAL;
CREATE OR REPLACE TYPE BODY termek_t AS
  ORDER MEMBER FUNCTION
    osszehasonlit(masik termek_t) RETURN INTEGER IS
  BEGIN
    IF SELF.ar < masik.ar THEN RETURN -1;
    ELSIF SELF.ar > masik.ar THEN RETURN 1;
    ELSE RETURN 0; END IF;
  END; END;
```

2. Öröklés

Szülőosztályból származtatott altípus hozható létre az UNDER kulcsszóval

Típushierarchia



Szintaxis

```oracle_plsql
-- 1. Szulotipus (!NOT FINAL kotelezo!)
CREATE OR REPLACE TYPE szemely_t AS OBJECT (
  nev   VARCHAR2(100),
  email VARCHAR2(200),
  MEMBER FUNCTION bemutatkozas RETURN VARCHAR2
) NOT FINAL;   -- kulcsszo az orokleshez
/
-- 2. Altipus az UNDER kulcsszoval
CREATE OR REPLACE TYPE alkalmazott_t
  UNDER szemely_t (
  beosztas VARCHAR2(100),
  fizetes  NUMBER,
  OVERRIDING MEMBER FUNCTION bemutatkozas
    RETURN VARCHAR2
) NOT FINAL;
-- 3. Masodik szint
CREATE OR REPLACE TYPE menedzser_t
  UNDER alkalmazott_t (
  reszleg VARCHAR2(100),
  keret NUMBER
);
```

- NOT FINAL kötelező a szülőben ha bármely típus örököl belőle.
- Egyszeres öröklés: csak EGY szülőtípus megengedett.

2b. OVERRIDING: metódusfelüldefiniálás és polimorfizmus

Az altípus felülírhatja a szülőtípus MEMBER FUNCTION-jét

Szülőtípus TYPE BODY

```oracle_plsql
CREATE OR REPLACE TYPE BODY szemely_t AS
  MEMBER FUNCTION bemutatkozas
  RETURN VARCHAR2 IS
  BEGIN
    RETURN 'Szemely: ' || SELF.nev;
  END bemutatkozas;
END;
/
```

Altípus TYPE BODY (OVERRIDING)

```oracle_plsql
CREATE OR REPLACE TYPE BODY alkalmazott_t AS
  OVERRIDING MEMBER FUNCTION bemutatkozas
  RETURN VARCHAR2 IS
  BEGIN
    RETURN 'Alkalmazott: ' || SELF.nev ||
           ' (' || SELF.beosztas || ')';
  END bemutatkozas;
END;
/
```

Polimorfizmus – ugyanaz a hívás, különböző eredmény

```oracle_plsql
CREATE TABLE szemelyek OF szemely_t;
INSERT INTO szemelyek VALUES (szemely_t     ('Kiss Peter',  'kiss@xy.hu'));
INSERT INTO szemelyek VALUES (alkalmazott_t ('Nagy Eva',    'nagy@xy.hu', 'Fejleszto', 500000));
INSERT INTO szemelyek VALUES (menedzser_t   ('Kovacs Adam', 'kov@xy.hu',  ,Menedzser', 800000, 'IT', 2000000));
COMMIT;
-- Polimorf hivas: mindenki a SAJAT bemutatkozas() fuggvenyet futtatja
SELECT s.bemutatkozas() FROM szemelyek s;
-- „Szemely: Kiss Peter”            -- szemely_t fuggvenye
-- „Alkalmazott: Nagy Eva (Fejleszto)”  -- alkalmazott_t fuggvenye
-- „Menedzser: Kovacs Adam(Menedzser), IT részleg”  -- menedzser_t fuggvenye
```

Polimorfizmus lényege: ugyanazon típushierarchia különböző példányain ugyanaz a metódushívás különböző kódot hajt végre

2c. IS OF és TREAT: típusvizsgálat és konverzió

Polimorf lekérdezéseknél lehetőségünk van az típusok ellenőrzésére (IS OF), vagy azok altípusként való kezelésére (TREAT)

IS OF – tipusvizsgalat

Megmutatja, hogy egy objektum egy adott típus példánya-e

```
-- Csak alkalmazottak (menedzserek is)
SELECT s.nev FROM szemelyek s
WHERE VALUE(s) IS OF (alkalmazott_t);

-- Csak a konkret menedzser tipusok altipusok nelkul
WHERE VALUE(s) IS OF (ONLY menedzser_t);

-- Tobb tipus OR logikával
WHERE VALUE(s) IS OF
  (alkalmazott_t, menedzser_t);
```

TREAT – tipuskonverzio (cast)

Szülőtípusú változót altípusként kezelünk, ha nem sikerül, NULL-t ad

```
-- Alkalmazott fizetesenek elerése
-- (szemely_t tipusban nincs fizetes!)
SELECT
  s.nev,
  TREAT(VALUE(s) AS alkalmazott_t).fizetes
    AS fizetes
FROM szemelyek s
WHERE VALUE(s) IS OF (alkalmazott_t);

-- Ha konverzio nem sikerül: NULL (nem hiba!)
```

Operátor	Cél	Visszatérési érték	Hibát dob, ha nem illeszkedik?
`IS OF (tipus_t)`	Típusellenőrzés	TRUE / FALSE	Nem
`TREAT(obj AS t)`	Típuskonverzió	Altípus példánya	Nem – NULL
`CAST(obj AS t)`	Erős konverzió	Altípus példánya	IGEN – hibaüzenet

Elmélet vége

..de nincs vége a jónak,
következik a gyakorlati rész